In [1]:
```
import pandas as pd
import numpy as np


df=pd.read_csv('loan_approval_dataset.csv')
df.head()

df.info()
df.keys()

df.columns=df.columns.str.strip()

df.columns
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4269 entries, 0 to 4268
Data columns (total 13 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   loan_id                   4269 non-null   int64
 1    no_of_dependents         4269 non-null   int64
 2    education                4269 non-null   object
 3    self_employed            4269 non-null   object
 4    income_annum             4269 non-null   int64
 5    loan_amount              4269 non-null   int64
 6    loan_term                4269 non-null   int64
 7    cibil_score              4269 non-null   int64
 8    residential_assets_value 4269 non-null   int64
 9    commercial_assets_value  4269 non-null   int64
 10   luxury_assets_value      4269 non-null   int64
 11   bank_asset_value         4269 non-null   int64
 12   loan_status              4269 non-null   object
dtypes: int64(10), object(3)
memory usage: 433.7+ KB
```

Out[1]:
```
Index(['loan_id', 'no_of_dependents', 'education', 'self_employed',
       'income_annum', 'loan_amount', 'loan_term', 'cibil_score',
       'residential_assets_value', 'commercial_assets_value',
       'luxury_assets_value', 'bank_asset_value', 'loan_status'],
      dtype='object')
```

In [2]:
```
df.drop(columns = ['loan_id'],inplace = True )
```

In [3]:
```
df.head()
```

Out[3]:

	no_of_dependents	education	self_employed	income_annum	loan_amount	l
0	2	Graduate	No	9600000	29900000	
1	0	Not Graduate	Yes	4100000	12200000	
2	3	Graduate	No	9100000	29700000	
3	3	Graduate	No	8200000	30700000	
4	5	Not Graduate	Yes	9800000	24200000	

In [4]:
```
df.shape
```

Out[4]:
```
(4269, 12)
```

In [5]:
```
df.isnull().sum()
```

Out[5]:
```
no_of_dependents            0
education                   0
self_employed               0
income_annum                0
loan_amount                 0
loan_term                   0
cibil_score                 0
residential_assets_value    0
commercial_assets_value     0
luxury_assets_value         0
bank_asset_value            0
loan_status                 0
dtype: int64
```

In [6]:
```
df['Assert'] = df['residential_assets_value'] + df['commercial_assets_value']
```

In [7]:
```
df.head()
```

Out[7]:

	no_of_dependents	education	self_employed	income_annum	loan_amount	l
0	2	Graduate	No	9600000	29900000	
1	0	Not Graduate	Yes	4100000	12200000	
2	3	Graduate	No	9100000	29700000	
3	3	Graduate	No	8200000	30700000	
4	5	Not Graduate	Yes	9800000	24200000	

In [8]:
```
df.drop(columns = ['residential_assets_value','commercial_assets_value','luxur
```

In [9]:
```
df.head()
```

Out[9]:

	no_of_dependents	education	self_employed	income_annum	loan_amount	l
0	2	Graduate	No	9600000	29900000	
1	0	Not Graduate	Yes	4100000	12200000	
2	3	Graduate	No	9100000	29700000	
3	3	Graduate	No	8200000	30700000	
4	5	Not Graduate	Yes	9800000	24200000	

In [10]:
```
df['education'].unique()
```

Out[10]:
```
array([' Graduate', ' Not Graduate'], dtype=object)
```

In [11]:
```
df['education']=df['education'].apply(lambda x:x.strip())
```

In [12]:
```
df['education'].unique()
```

Out[12]:
```
array(['Graduate', 'Not Graduate'], dtype=object)
```

In [13]:
```
df['education']=df['education'].replace(['Graduate','Not Graduate'],[1,0])
```

```
C:\Users\pc10\AppData\Local\Temp\ipykernel_13896\2487469019.py:1: FutureWarnin
g: Downcasting behavior in `replace` is deprecated and will be removed in a fut
ure version. To retain the old behavior, explicitly call `result.infer_object
s(copy=False)`. To opt-in to the future behavior, set `pd.set_option('future.n
o_silent_downcasting', True)`
  df['education']=df['education'].replace(['Graduate','Not Graduate'],[1,0])
```

In [14]:
```
df.head()
```

Out[14]:

	no_of_dependents	education	self_employed	income_annum	loan_amount	l
0	2	1	No	9600000	29900000	
1	0	0	Yes	4100000	12200000	
2	3	1	No	9100000	29700000	
3	3	1	No	8200000	30700000	
4	5	0	Yes	9800000	24200000	

In [15]:
```
df['loan_status'].unique()
```

Out[15]:
```
array([' Approved', ' Rejected'], dtype=object)
```

In [16]:
```
df['loan_status']=df['loan_status'].apply(lambda x:x.strip())
```

In [17]:
```
df['loan_status'].unique()
```

Out[17]:
```
array(['Approved', 'Rejected'], dtype=object)
```

In [18]:
```
df['loan_status']=df['loan_status'].replace(['Approved','Rejected'],[1,0])
```

```
C:\Users\pc10\AppData\Local\Temp\ipykernel_13896\294652728.py:1: FutureWarning:
Downcasting behavior in `replace` is deprecated and will be removed in a future
version. To retain the old behavior, explicitly call `result.infer_objects(cop
y=False)`. To opt-in to the future behavior, set `pd.set_option('future.no_sile
nt_downcasting', True)`
  df['loan_status']=df['loan_status'].replace(['Approved','Rejected'],[1,0])
```

In [19]:
```
df.head()
```

Out[19]:

	no_of_dependents	education	self_employed	income_annum	loan_amount	l
0	2	1	No	9600000	29900000	
1	0	0	Yes	4100000	12200000	
2	3	1	No	9100000	29700000	
3	3	1	No	8200000	30700000	
4	5	0	Yes	9800000	24200000	

In [20]:
```
df['self_employed'].unique()
```

Out[20]:
```
array([' No', ' Yes'], dtype=object)
```

In [21]:
```
df['self_employed']=df['self_employed'].apply(lambda x:x.strip())
```

In [22]:
```
df['self_employed'].unique()
```

Out[22]:
```
array(['No', 'Yes'], dtype=object)
```

In [23]:
```
df['self_employed']=df['self_employed'].replace(['Yes','No'],[1,0])
```

```
C:\Users\pc10\AppData\Local\Temp\ipykernel_13896\1362126969.py:1: FutureWarnin
g: Downcasting behavior in `replace` is deprecated and will be removed in a fut
ure version. To retain the old behavior, explicitly call `result.infer_object
s(copy=False)`. To opt-in to the future behavior, set `pd.set_option('future.n
o_silent_downcasting', True)`
  df['self_employed']=df['self_employed'].replace(['Yes','No'],[1,0])
```

In [24]:
```
df.head()
```

Out[24]:

	no_of_dependents	education	self_employed	income_annum	loan_amount	l
0	2	1	0	9600000	29900000	
1	0	0	1	4100000	12200000	
2	3	1	0	9100000	29700000	
3	3	1	0	8200000	30700000	
4	5	0	1	9800000	24200000	

In [25]:
```
X=df.drop(columns=['loan_status'])
y=df['loan_status']
```

In [26]:
```
from sklearn.model_selection import train_test_split
```

In [27]:
```
x_train,x_test,y_train,y_test = train_test_split(X,y,test_size=0.2,random_stat
```

In [28]:
```
x_train.shape, x_test.shape, y_train.shape, y_test.shape
```

Out[28]:
```
((3415, 8), (854, 8), (3415,), (854,))
```

In [29]:
```
from sklearn.preprocessing import StandardScaler
```

In [30]:
```
scaler = StandardScaler()
```

In [31]:
```
x_train_scaled = scaler.fit_transform(x_train)
```

In [32]:
```
x_test_scaled = scaler.transform(x_test)
```

In [33]:
```
from sklearn.linear_model import LogisticRegression
```

In [34]:
```
model = LogisticRegression()
```

In [35]:
```
model.fit(x_train_scaled, y_train)
```

Out[35]:
```
▾ LogisticRegression
LogisticRegression()
```

In [36]:
```
model.score(x_test_scaled, y_test)
```

Out[36]:
```
0.905152224824356
```

In [37]:
```
model.predict(x_test_scaled)
```

Out[37]:
```
array([0, 1, 0, 1, 1, 1, 1, 0, 1, 0, 1, 1, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0,
       0, 0, 1, 1, 1, 1, 1, 0, 1, 0, 1, 1, 0, 0, 1, 1, 1, 0, 0, 1, 1, 1,
       0, 1, 1, 1, 1, 0, 1, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 0, 0, 1, 1, 0, 0, 0, 1, 0, 0, 1, 1,
       0, 0, 1, 0, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 1, 1, 0, 1, 0, 1, 1, 1,
       1, 1, 0, 0, 1, 0, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 0, 0, 1, 1, 0,
       1, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1, 1, 1, 1, 0, 0,
       1, 1, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 0, 1, 1, 0, 1,
       1, 0, 1, 0, 0, 1, 0, 0, 0, 1, 1, 1, 0, 1, 1, 0, 0, 0, 1, 1, 0, 1,
       1, 1, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 0, 0, 1, 1, 1, 0, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 0, 1, 0, 1, 0, 0, 0, 1, 1, 1, 1, 0, 1, 0, 0,
       1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 1, 0, 0, 0, 1, 0, 1, 1, 1, 1, 0,
       1, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 0, 0, 1, 1, 1, 1,
       1, 0, 0, 1, 1, 1, 0, 1, 0, 1, 1, 0, 0, 0, 0, 0, 1, 1, 0, 1, 1, 1,
       1, 1, 1, 0, 1, 0, 1, 1, 0, 1, 0, 1, 1, 1, 1, 1, 0, 0, 1, 0, 0, 0,
       1, 0, 1, 1, 1, 0, 0, 1, 1, 1, 1, 1, 0, 1, 1, 0, 0, 1, 1, 0, 1, 1,
       0, 0, 1, 1, 1, 0, 0, 0, 1, 0, 1, 1, 0, 0, 0, 1, 1, 1, 1, 0, 1, 0,
       0, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0,
       1, 1, 0, 1, 0, 1, 0, 1, 1, 1, 0, 0, 1, 1, 0, 1, 1, 0, 1, 0, 0, 0,
       0, 1, 0, 0, 0, 1, 1, 1, 0, 1, 1, 1, 0, 1, 1, 1, 0, 0, 0, 0, 1, 1,
       0, 1, 1, 0, 1, 0, 1, 1, 1, 1, 0, 1, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1,
       0, 1, 1, 1, 0, 0, 0, 1, 1, 1, 1, 0, 1, 1, 0, 0, 0, 1, 1, 1, 0, 1,
       1, 1, 1, 1, 0, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 1, 1, 1, 1, 0, 1, 1,
       0, 1, 1, 1, 0, 1, 0, 0, 0, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 1, 1, 0,
       1, 1, 1, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 0, 1, 1, 0, 1, 1, 1, 1, 1,
       0, 1, 1, 1, 1, 1, 0, 1, 1, 0, 1, 1, 1, 1, 0, 0, 1, 0, 1, 0, 0, 1,
       1, 0, 1, 0, 1, 0, 1, 1, 1, 0, 0, 1, 1, 0, 1, 0, 0, 1, 1, 1, 0, 1,
       1, 1, 1, 0, 1, 1, 0, 1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 0, 0, 1, 1, 1,
       0, 1, 0, 0, 0, 1, 1, 1, 1, 1, 1, 0, 1, 1, 0, 0, 1, 1, 1, 0, 1, 1,
       0, 1, 1, 0, 0, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 1, 0, 1, 1,
       0, 1, 1, 0, 0, 1, 0, 1, 1, 1, 1, 0, 1, 1, 0, 0, 0, 1, 1, 1, 0, 0,
       1, 0, 1, 1, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 0, 0, 1, 1, 1, 0, 1, 0,
       0, 1, 0, 1, 1, 1, 1, 0, 0, 1, 1, 0, 1, 0, 1, 1, 0, 0, 1, 0, 1, 0,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 0, 1, 1, 1, 0, 1, 0, 1, 1, 1, 0,
       0, 1, 1, 0, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 0, 0, 1, 1, 0, 0, 1,
       0, 1, 1, 0, 1, 1, 1, 0, 1, 1, 1, 1, 0, 1, 0, 0, 1, 0, 1, 1, 1, 1,
       0, 0, 0, 0, 1, 0, 1, 1, 1, 0, 1, 1, 1, 0, 1, 1, 1, 0, 1, 0, 0, 1,
       0, 1, 1, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1, 1, 0, 1, 0, 0, 0, 1, 1, 0,
       1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1])
```

In [38]:
```
pred_data = pd.DataFrame([['2','1','0','9600000','29900000','12','778','507000
```

In [39]:
```
pred_data = scaler.transform(pred_data)
```

In [40]:
```
model.predict(pred_data)
```

Out[40]:
```
array([1])
```

In [41]:
```
import pickle as pk
```

In [42]:
```
pk.dump(model,open('model.pkl','wb'))
```

In [43]:
```
pk.dump(scaler,open("scaler.pkl",'wb'))
```

Python Code

```python
import streamlit as st
import pandas as pd
import pickle as pk

st.set_page_config(
    page_title="Loan Prediction App",
    page_icon="",
    layout="centered"
)

model=pk.load(open('model.pkl','rb'))
scaler=pk.load(open('scaler.pkl','rb'))

st.header('Loan Prediction App')

no_of_dep = st.slider('Choose No of dependents', 0, 10)
grad = st.selectbox('Choose Education',['Graduated','Not Graduated'])
self_emp = st.selectbox('Self Emoployed ?',['Yes','No'])
Annual_Income = st.slider('Choose Annual Income', 0, 10000000)
Loan_Amount = st.slider('Choose Loan Amount', 0, 10000000)
Loan_Dur = st.slider('Choose Loan Duration', 0, 20)
Cibil = st.slider('Choose Cibil Score', 0, 1000)
Assert = st.slider('Choose Assert', 0, 10000000)

if grad =='Graduated':
    grad_s = 1
else:
    grad_s = 0

if self_emp =='Yes':
    emp_s =1
else:
    emp_s = 0

if st.button("Predict"):
    pred_data =pd.DataFrame([[no_of_dep,grad_s,emp_s,Annual_Income,Loan_Amount,Loan_Dur,Cibil,Assert]],columns=['no_of_dependents','education','self_employed','income_annum','loan_amount','loan_term','cibil_score','Assert'])
    pred_data = scaler.transform(pred_data)
    predict = model.predict(pred_data)
    if predict[0] == 1:
        st.markdown('Loan Is Approved')
    else:
        st.markdown('Loan Is Rejected')
```

Loan Prediction App

Choose No of dependents

0

Choose Education

Graduated

Self Emoployed ?

Yes

Choose Annual Income

0

Choose Loan Amount

0

Choose Loan Duration

0

Choose Cibil Score

0

Choose Assert

0

Predict

Loan Prediction App

Choose No of dependents

2

Choose Education

Graduated

Self Emoployed ?

Yes

Choose Annual Income

4711129

Choose Loan Amount

3453925

Choose Loan Duration

8

Choose Cibil Score

685

0 1000

Choose Assert

4595524

Predict

Loan Is Approved

Loan Prediction App

Choose No of dependents

7

Choose Education

Graduated

Self Emoployed ?

Yes

Choose Annual Income

3078208

Choose Loan Amount

6098388

Choose Loan Duration

9

Choose Cibil Score

421

Choose Assert

2326776

Predict

Loan Is Rejected